Day 15 Homework: Alerts and a Runbook

By Sadasivam Mithran

The big picture

My chatbot now has a dashboard that can raise an alarm when something looks wrong, like too many failed answers. For this homework I (1) wrote the code that decides when to raise an alarm, (2) picked four alarm rules and set off two fake outages on purpose to check the alarms really go off, and (3) wrote a one-page guide, called a runbook, that tells whoever is on call what to do for each alarm. All my tests pass (348 in total, and the 5 for this task).

Task 1: The code that raises the alarm

The function **check_alerts()** looks at the dashboard numbers and at a list of rules. A rule says, for example, 'if the error rate is above 5%, raise an alarm'. For each rule the function does three simple things: it skips the rule if there have been too few requests to judge fairly, skips it if the number it needs is missing, and otherwise raises the alarm only if the number is bigger than the limit. Each alarm that goes off comes back with its name, the current value, the limit, and the first thing to do about it.

```
        def check_alerts(summary: dict, rules: list[dict]) -> list[dict]:
            fired = []
        for rule in rules:
            if summary.get("requests", 0) < rule.get("min_requests", 1):
                continue  # too few requests to judge
            value = summary.get(rule["metric"])
            if value is None or value <= rule["above"]:
                continue  # missing, or within limits
            fired.append(
                {
                    "name": rule["name"],
                    "metric": rule["metric"],
                    "value": value,
                    "above": rule["above"],
                    "action": rule.get("action", ""),
                }
            )
        return fired
```

Task 2: My four rules

I chose four rules that measure things users would actually feel. Fewer, useful rules are better than many noisy ones.

Rule	Goes off when	Why I picked it
Too many errors	more than 5% of requests fail (needs at least 10 requests)	Failed answers mean unhappy users
Slow answers	the slowest 5% of answers take over 20 seconds (at least 10 requests)	Users give up if they wait too long
Running on the fallback model	more than 2 answers came from the backup model (at least 5 requests)	Early warning: the main model is struggling but users are still being served
Spending more than planned	estimated cost goes over $1.00	Catches a runaway loop or a flood of requests

One threshold I changed. I first set the slow-answers limit to 8 seconds, as in the example. But when I ran the real chatbot normally, the slowest answers took up to 15 seconds, so the alarm went off even though nothing was wrong. I raised the limit to 20 seconds. A good alarm should only go off when something is actually broken.

The two outage drills

To prove the alarms work, I broke the chatbot on purpose. I used the fake chatbot, which costs nothing, and cleared the request log before each drill. Each time I sent the 10 test questions twice (20 requests).

Drill 1: the main model is down. The chatbot switched to its backup model and still answered all 20 questions, 14 of them using the backup. Nobody saw an error, but the 'Running on the fallback model' alarm went off, which is exactly what it is for. Here is what the dashboard reported:


```
[
 {
  "name": "Running on the fallback model",
  "metric": "fallbacks",
  "value": 14,
  "above": 2,
  "action": "The primary model is failing or rate-limiting: check the primary provider's status page. Runbook sect
 }
]
```


Drill 2: every model is down. Now there was no backup left. 14 questions got an error message (503, 'try again later'). The 6 questions the chatbot declines anyway were still answered, because they never need a model. The error rate hit 70% and the 'Too many errors' alarm went off:


```
[
 {
  "name": "Too many errors",
  "metric": "error_rate",
  "value": 0.7,
  "above": 0.05,
  "action": "Open Phoenix, find a failed trace and read which step failed; check both providers. Runbook section 3
 }
]
```


After the drills I restarted the chatbot normally and ran a real 10-question check: no errors, no alarms, typical answer time about 5 seconds.

What I learned

1. The fallback alarm warned me about the main model failing before any user saw a problem. 2. My first slow-answers limit was too tight, so testing against a real, healthy run was important. 3. The cost number was showing $0.00 for my real model because the code did not know its price. I added the price (an estimate of $0.60 per million tokens; I should double-check it on OpenRouter), and a real run now shows about $0.004 for 10 questions. 4. The cost alarm cannot be tested with the fake chatbot, because the fake one costs nothing.

Extra work: four earlier class activities were not finished, and the drills could not run without them (answering questions, the 'is it ready' check, tracing, and retry-then-fallback). I finished all four and their tests now pass.

Task 3: The runbook

The runbook is a one-page guide for someone who has never seen the chatbot. It explains what the service does, how to start, stop and check it, what to do for each of the four alarms (what it means, three first steps, and how you know it is fixed), problems I already know about, and who to contact. It contains no passwords or keys. The full text is below, and it is also saved in the project as docs/RUNBOOK.md.

One page. Written for the person on call at 3 a.m. who has never seen this service before. Run commands from the project folder (`HW 15`).

1. What the service does

Staff ask IT policy questions at the chat page. It answers only from the class policy documents, cites its sources, and declines when the documents don't cover the question. If it is down, staff get no answers; nothing is lost.

Item -- Value

Chat page -- http://localhost:8000

Dashboard -- http://localhost:8000/dashboard (alerts are listed at the top; raw: `GET /metrics`)

Traces -- http://localhost:6006 (Phoenix, project helpdesk-chatbot)

Primary model / fallback model -- `deepseek/deepseek-chat` (`LLM_MODEL` in `.env`) / `openai/gpt-4.1-mini` (`FALLBACK_MODEL`), both via OpenRouter

Depends on -- OpenRouter + `OPENROUTER_API_KEY`; Ollama on the host (`nomic-embed-text`, port 11434) to embed questions; the index in `data/chroma` (62 chunks, baked into the image); Phoenix container

2. Start, stop, check

Task -- Command

Start -- `docker compose up -d` (add `--build` after a code change)

Stop -- `docker compose down`

Is it alive? -- `curl localhost:8000/health` → `{"status":"ok"}`

Is it ready? What is missing? -- `curl localhost:8000/ready` → 200 `ready:true`, or 503 with the failing check named

Read the log -- `docker compose logs --tail 100 chatbot`; one line per request: `logs/requests.jsonl`

Roll back to the previous image -- `docker tag helpdesk-chatbot:previous helpdesk-chatbot && docker compose up -d --no-build` (before each deploy: `docker tag helpdesk-chatbot helpdesk-chatbot:previous`)

No models / no cost (testing) -- `CHAT_FAKE=1 docker compose up -d`; practise outages with `SIMULATE_OUTAGE=primary` or `=all`

3. Alerts: what each one means and what to do first

Numbers cover the last 500 logged requests, so an alert clears only after enough good requests push the bad ones out (or after `rm logs/requests.jsonl` once the cause is fixed).

3.1 Too many errors (error_rate above 0.05, at least 10 requests)

- **What it usually means:** both models fail, so users get 503. Drill 2 (every model down) gave 0.7.

- **First three steps:** 1. Phoenix: open a failed `ask` trace and read which step raised. 2. `curl localhost:8000/ready`; check OpenRouter's status page and that the key has credit. 3. `docker compose logs --tail 100 chatbot` for the error text.

- **Fixed when:** `error_rate` is back under 0.05 on the dashboard and a test question answers with 200.

3.2 Slow answers (p95_seconds above 20, at least 10 requests)

- **What it usually means:** the model is slow or retrying. Normal real runs measured p50 4.9-6.4 s and p95 6-15 s (deepseek, depending on provider speed), so 20 s is deliberately above normal. I first tried 8 s, and it fired on a healthy system. A fallback only adds about 1.5 s, so this rule catches much worse cases.

- **First three steps:** 1. Phoenix: find the slowest span (`open_model_stream` or `generate`). 2. Is the fallback in use (see 3.3)? 3. Check Ollama (`/ready`) and the OpenRouter status page.

- **Fixed when:** p95 is under 20 s again over a fresh run of `python -m scripts.send_questions`.

3.3 Running on the fallback model (fallbacks above 2, at least 5 requests)

- **What it usually means:** the primary provider is failing or rate-limiting us. Users still get answers. Drill 1 gave 14 fallbacks and no errors.

- **First three steps:** 1. Phoenix: failed ChatCompletion spans before each success? 2. Check the primary provider's status page. 3. Spot-check three fallback answers: citations, tone.

- **Fixed when:** new answers show the primary model again on the dashboard (`by_model`) and `SIMULATE_OUTAGE` is not set (`docker compose exec chatbot env | grep SIMULATE`).

3.4 Spending more than planned (cost_usd above 1.0)

- **What it usually means:** a loop or a flood of requests. A normal 20-question run costs a few cents. Cost is only estimated for models listed in `PRICES` in `app/monitoring.py`: an unlisted model shows $0.00.

- **First three steps:** 1. Dashboard recent list: one caller or one question repeated? 2. Check the OpenRouter usage page for the true spend. 3. If it is a flood, `docker compose down` until the source is found.

- **Fixed when:** requests per minute are back to normal and spend stops growing. This rule cannot be drilled with the fake chatbot (it costs nothing).

4. Known failures

Symptom -- Likely cause -- Fix

/ready says ollama failed -- Ollama is not running on the laptop, or the container cannot reach `host.docker.internal:11434` -- Start Ollama; `docker compose restart chatbot`. `/health` stays ok, only `/ready` fails

/ready says index failed -- `data/chroma` missing or empty when the image was built -- `python -m scripts.build_index_vision`, then `docker compose up -d --build`

Every answer comes from the fallback model -- Primary provider down, rate limited, or `SIMULATE_OUTAGE=primary` left set -- See 3.3; restart with no `SIMULATE_OUTAGE`

Dashboard cost shows $0.00 on real answers -- Model missing from `PRICES` -- Add its price per million tokens in `app/monitoring.py`, rebuild

Alert will not clear after a fix -- Old bad requests still in the last 500 -- Wait, or archive and delete `logs/requests.jsonl`

Answers are empty or declined for everything -- Index is not matching the question model (embeddings) -- Check `/ready`; rebuild the index with the same `EMBED_MODEL`

5. Escalation

1. The service owner (Mithran Sadasivam, mithran.sadasivam@gmail.com): if not fixed within 30 minutes, or any time `/ready` stays 503 after the steps above.

2. The model provider (OpenRouter support) if their status page shows no incident but every call fails.

Tell them: the alert name and value, the time it started (first failing line in `logs/requests.jsonl`), one `request_id` of a failed request, and its Phoenix trace link. Never paste `.env` or the API key.